

Design: Proper Fix for P2 TopicPageDto Crash on IA WPO-* Topics

Crashlytics issue: 8cde0ac208b3 (NON_FATAL) **Symptom:** Internet Archive topic requests (IDs of the form WPO-*, NASAJPL-*, etc.) cause deserialization failures on the Android client because the /topic2/:id endpoint returns empty or zero-value data for non-rutracker IDs. **Partial fix shipped:** version code 1073 added a commentsPage default to TopicPageDto. **Status of other 5 fields:** still crashable under conditions described below. **Date:** 2026-06-24 **Author:** investigation agent (read-only; no code was changed)

1. Root Cause — Server-Side Finding

1.1 The architectural lock (handlers.go:77)

```
// lava-api-go/internal/handlers/handlers.go:77
var _ ScraperClient = (*rutracker.Client)(nil)
```

This compile-time assertion means the ScraperClient interface — which the entire handler layer depends on — can ONLY be satisfied by *rutracker.Client. There is no provider dispatch in this handler package: it is rutracker-only by design.

The /topic2/:id route is registered at handlers.go:115:

```
router.GET("/topic2/:id", topic.GetTopicPage)
```

And the handler dispatches unconditionally to the rutracker scraper at topic.go:131:

```
page, err := h.scraper.GetTopicPage(c.Request.Context(), id,
    pagePtr, cookie)
```

No ID format validation exists before this call. WPO-* IDs pass through to the rutracker scraper as-is.

1.2 What the rutracker scraper does with a WPO-* ID (rutracker/topic.go:39-93)

ParseTopicPage at rutracker/topic.go:39 runs goquery selectors on the HTML returned by rutracker.org/viewtopic.php?t=WPO-123456. The page returned is a rutracker error or index page (not a topic page), so every selector misses:

Selector	Expected match	Actual result for WPO-*
#topic-title	topic anchor with ?t=<id>	no match → id = ""
#topic-title text	topic title text	no match → title = ""
tbody[id^=post]	first post row	no match → author = nil
.nav.w100.pad_2 a:last	breadcrumb category	no match → category = nil
parseTorrentData goquery	torrent info block	no match → torrentData = nil
#pagination > tbody...	page count	no match → page=1, pages=1, posts=[]

The function returns (&gen.TopicPageDto{Id:"", Title:"", Author:nil, Category:nil, CommentsPage:{1,1,[]}, TorrentData:nil}, nil) — a zero-value struct with a nil error.

There is also a false-positive predicate at rutracker/topic_predicates.go:17-21: isTopicExists() checks for three Russian-language error strings; a rutracker error page for WPO-* contains none of them, so it returns true (topic "found"), and the zero-value struct flows out as a valid HTTP 200 response.

1.3 Go struct serialization (gen/server/api.gen.go:1012-1019)

```
type TopicPageDto struct {
    Author      *AuthorDto
        `json:"author"`           // nil → "author":null
    Category    *CategoryDto
        `json:"category"`         // nil → "category":null
    CommentsPage TopicPageCommentsDto `json:"commentsPage"` //
        value → present
    Id          string           `json:"id"`           // ""
        → "id":""
    Title       string           `json:"title"`        // ""
        → "title":""
    TorrentData *TorrentDataDto
        `json:"torrentData"`      // nil → "torrentData":null
}
```

None of these fields carry `omitempty`. Go's `encoding/json` therefore serializes ALL six fields. The HTTP 200 response body for a WPO-* request is:

```
{"author":null,"category":null,"commentsPage":{"page":1,"pages":1,"posts":
    []},"id":"","title":"","torrentData":null}
```

All six keys ARE present in the current codebase. The Kotlin client should NOT be receiving an id-absent response today. However:

- Responses are **cached with a 5-minute TTL** (`topicGroupTTL`, `topic.go:42`). A cached response generated by an older server version that DID have `omitempty` on string fields could produce absent keys for up to 5 minutes after a server upgrade.
- The **Ktor proxy** (:proxy module, the other backend the Android client can connect to) has independent Kotlin serialization code with its own behavior. If an IA topic request hits the Ktor backend, the result may differ.
- The 1073 `commentsPage` default would have been unnecessary if the server always sent `commentsPage`. Its presence confirms there was (or remains) at least one scenario where the key is absent.

The root cause is the same regardless of the exact absent-field path: **the server does not know how to serve IA topics from /topic2**, and any response it returns for WPO-* IDs contains semantically empty data.

1.4 The IA adapter that CAN serve real data (archiveorg/provider.go:118-137)

```
// lava-api-go/internal/archiveorg/provider.go:118-137
func (a *ProviderAdapter) GetTopic(ctx context.Context, id
    string, ...) (*provider.TopicResult, error) {
    tr, err := a.client.Topic(ctx, id)
    // ...
    return &provider.TopicResult{
        Provider:    a.ID(),
        ID:          tr.ID,
        Title:       tr.Title,
        Description: tr.Description,
        Files:       files,
    }, nil
}
```

This method fetches real data from `archive.org/metadata/{identifier}` and returns a populated `TopicResult` with ID, Title, Description, and Files. It is reachable via `/v1/archiveorg/topic/:id` (the v1 provider system), but **not from /topic2/:id** — that route has no provider registry wiring.

2. Client-Side Finding

2.1 TopicPageDto field requirements (TopicPageDto.kt:7-14)

```
// core/network/api/src/main/kotlin/lava/network/dto/topic/
// TopicPageDto.kt:7-14
@Serializable
data class TopicPageDto(
    val id:
        String,                                // non-nullable, NO default →
        key required in JSON
    val title:
        String,                                // non-nullable, NO default → key
        required in JSON
    val author: AuthorDto?,                    // nullable, NO default →
        key still required
    val category: CategoryDto?,                // nullable, NO default →
        key still required
    val torrentData: TorrentDataDto?,          // nullable, NO default →
        key still required
    val commentsPage: TopicPageCommentsDto =
        TopicPageCommentsDto(...), // has default ← 1073 fix
)
```

In kotlinx.serialization, a nullable field (AuthorDto?) WITHOUT an explicit = null default is still REQUIRED to be present in the JSON. Absence of the key — not the presence of null — is what throws MissingFieldException. The value "author":null deserializes correctly; the absent key {} does not.

This means: if any server path returns a response where author, category, or torrentData keys are absent (as opposed to present-and-null), the client throws a NON_FATAL on deserialization.

For id and title (non-nullable, no default): absent OR null would both fail.

2.2 What the topic screen actually renders

Tracing TopicViewModel.kt:89-126 and TopicScreen.kt:

Field	Where used	Effect if empty/null
id	Favorites add/remove, navigation back-stack key, loadTopic identifier	id="" → favorites written with "" key; may cause silent collision
title		

Field	Where used	Effect if empty/null
	App bar text (TopicContent.Topic(topic.title))	Blank app bar
author	NOT displayed in topic header	No visual impact
category	NOT displayed in topic header	No visual impact
torrentData	Gates the download section (if (torrentData != null))	No download section shown
commentsPage	Comment list, pagination	Empty comment list

author and category are not displayed in the topic header — they only appear in individual post rows within commentsPage.posts. Since IA topics have no posts (posts=[]), those fields are effectively unreachable from the UI for IA items.

Hard UI requirements for a minimally functional IA topic screen:

- id: non-empty (navigation/favorites correctness)
- title: non-empty (app bar)
- torrentData: null is acceptable (download section gracefully hidden)
- author, category: null is acceptable (not rendered)
- commentsPage: default is acceptable (1073 fix covers this)

3. Options Analysis

Option A — Server-side IA dispatch in /topic2 (RECOMMENDED)

Add provider-awareness to the /topic2 handler. When the ID is non-numeric (rutracker IDs are pure integers; WPO-* IDs contain letters), dispatch to the appropriate provider adapter instead of the rutracker scraper.

For WPO-* specifically, call archiveorg.GetTopic (which already exists at internal/archiveorg/provider.go:118) and map the result to gen.TopicPageDto:

TopicPageDto.Id	← TopicResult.ID	(e.g. "WPO-123456")
TopicPageDto.Title	← TopicResult.Title	(real title from archive.org metadata)

TopicPageDto.Author	← nil	(IA has no author concept)
TopicPageDto.Category	← nil	(IA has no rutracker-style category)
TopicPageDto.TorrentData	← nil	(IA download handled via /download/:id)
TopicPageDto.CommentsPage	← {1,1,[]}	(IA has no comments)

Pros:

- Real data (id, title) returned — topic screen shows meaningful content
- No client changes required
- §6.J compliant: rutracker topics are unaffected (numeric-ID gate is exact)
- IA data genuinely exists on archive.org and is already fetched by the existing adapter

Cons:

- Requires modifying the handler layer's ScraperClient interface or adding a separate provider dispatch path alongside ScraperClient
- The handlers.go:77 compile-time lock must be relaxed or a parallel injection path added for the IA adapter

Option B — Dedicated sparse DTO for IA topics

Define a separate IaTopicPageDto on the server with only id and title, and a separate client DTO. Return it from a new /topic2ia/:id route.

Cons:

- Client must know at request time whether to call /topic2 or /topic2ia
- Requires client-side changes AND server-side changes
- Two code paths to maintain
- NOT recommended

Option C — Client-side defaults only

Add = "" to id and title, = null to author, category, torrentData.

Cons — §6.J VIOLATION:

A test that passes while the feature is broken is the most expensive kind of test.

Adding `= null` default to `author` in `TopicPageDto` means a genuinely malformed rutracker response (server bug returning absent `author` key for a NORMAL topic) would silently succeed. The serialization exception that currently surfaces the bug would be swallowed. This is the exact "masking normal-topic bugs" scenario prohibited by §6.J.

Additionally, `id = ""` would allow the screen to render with a blank app bar and favorites written with an empty-string key — a worse UX than an explicit error.

Option C is rejected on §6.J grounds.

Option D — Client skips `/topic2` for WPO-* IDs

Detect the WPO-* ID pattern on the client before calling `/topic2`, and navigate to a different screen or show a "not available" state.

Cons:

- Client-side heuristic for ID format matching (fragile; IA adds new prefixes)
 - User cannot view the topic at all — worse than Option A which shows real title + empty comment list
 - Pushes provider detection logic into the Android client
 - NOT recommended
-

4. Recommendation

Option A — Server-side dispatch using the existing archiveorg adapter.

Rationale:

1. The data exists: `archiveorg.GetTopic` at `internal/archiveorg/provider.go:118` already fetches real ID and Title from `archive.org/metadata/{identifier}`.
 2. Zero client changes needed to fix the crash and show a meaningful topic title.
 3. §6.J compliant: the numeric-ID gate (`strconv.Atoi(id)` fails for "WPO-*") is exact and does not affect rutracker paths at all.
 4. Extensible: the dispatch table can accept new non-numeric ID prefixes as more providers are added.
-

5. Concrete Change List for Option A

5.1 Immediate hotfix (can ship independently, zero risk)

In `lava-api-go/internal/handlers/topic.go:GetTopicPage`, add a numeric-ID guard before the scraper dispatch:

```
// Before dispatching to rutracker, validate the ID is a pure
// integer.
// WPO-*, NASAJPL-*, and other provider IDs contain letters.
if _, err := strconv.Atoi(id); err != nil {
    writeJSON(c, http.StatusNotFound, gin.H{})
    return
}
```

This stops the zero-value DTO from ever reaching the client for non-rutracker IDs. The client receives HTTP 404, which the existing error handling surfaces as a topic-not-found state rather than a silent deserialization failure.

Import `strconv` is already present in `topic.go:29`.

5.2 Full fix — Provider dispatch in `/topic2`

Step 1: Inject the IA adapter into the topic handler.

Add an `IaProvider` field to `TopicHandler` (or a new `ProviderTopicClient` interface accepting `GetTopic(ctx, id) (*gen.TopicPageDto, error)`). Wire it from `handlers.go` where the provider registry is already built.

Step 2: ID-format dispatch in `GetTopicPage`.

```
func (h *TopicHandler) GetTopicPage(c *gin.Context) {
    id := c.Param("id")
    // Rutracker IDs are pure integers. Non-numeric → provider
    // dispatch.
    if _, err := strconv.Atoi(id); err != nil {
        h.getTopicPageFromProvider(c, id)
        return
    }
    // ... existing rutracker path unchanged ...
}

func (h *TopicHandler) getTopicPageFromProvider(c *gin.Context,
    id string) {
    if !strings.HasPrefix(id, "WPO-") {
        // Future prefixes: extend here.
        writeJSON(c, http.StatusNotFound, gin.H{})
        return
    }
}
```



```

}
tr, err := h.iaAdapter.GetTopic(c.Request.Context(), id, 0,
    provider.Credentials{})
if err != nil {
    writeUpstreamError(c, err)
    return
}
page := &gen.TopicPageDto{
    Id:      tr.ID,
    Title:   tr.Title,
    // Author, Category, TorrentData intentionally nil – IA
    // has no equivalent.
    CommentsPage: gen.TopicPageCommentsDto{Page: 1, Pages: 1,
        Posts: []gen.PostDto{}},
}
body, _ := json.Marshal(page)
_ = h.cache.Set(c.Request.Context(), key, body, topicGroupTTL)
c.Data(http.StatusOK, "application/json", body)
}

```

Step 3: Update the compile-time lock.

The `var _ ScraperClient = (*rutracker.Client)(nil)` assertion at `handlers.go:77` must remain. The IA adapter is NOT added to `ScraperClient`; it is injected as a separate interface field. This preserves the `rutracker`-only guarantee on the existing 14 methods.

Step 4: Add contract test (§6.A mandate).

Following the `lava-api-go/tests/contract/` pattern: a new `topic_provider_dispatch_test.go` that exercises the ID-dispatch logic with a fake IA adapter, verifying:

- Pure-integer ID → `rutracker` path
- WPO-* ID → IA path → populated `id` + `title` in response JSON
- Unknown-prefix non-numeric ID → 404

6. §6.O Closure Requirements

When the fix commits land, per §6.O these items are required before closing in the Firebase console:

1. **Validation test (unit level).** A JUnit test (or Go test for the handler) that deserializes the real WPO-* payload shape and asserts on `id` and `title`.
2. **Challenge Test (end-to-end).** A Compose UI Challenge Test that opens an IA topic by WPO-* ID and asserts the topic title is rendered (non-empty).

3. **Closure log.** File at `.lava-ci-evidence/crashlytics-resolved/2026-06-24-8cde0ac-topicpagedto-ia-dispatch.md`.

7. Reproduce-First Test Plan (§6.T.1 mandate)

Per §6.T.1, the test **MUST** be written first and confirmed failing before the fix.

7.1 Go-level handler test (server side)

File: `lava-api-go/internal/handlers/topic_provider_dispatch_test.go`

Reproduce the zero-value-DTO scenario:

```
func TestGetTopicPage_WP0_ReturnsRealData(t *testing.T) {
    // Fake scraper that returns zero-value DTO for any ID
    // (current behavior).
    fakeScraper := &fakeScraperReturningZero{}
    // Fake IA adapter that returns real data for WP0-*.
    fakeIA := &fakeIAAdapter{id: "WP0-123456", title: "Apollo 11
        Mission Report"}

    h := &TopicHandler{scraper: fakeScraper, iaAdapter: fakeIA,
        cache: &nopCache{}}
    w := performGet(h, "/topic2/WP0-123456")

    var dto gen.TopicPageDto
    require.NoError(t, json.Unmarshal(w.Body.Bytes(), &dto))

    // PRIMARY assertions on user-visible fields:
    assert.Equal(t, "WP0-123456", dto.Id)           // MUST fail before
        fix (id="" currently)
    assert.Equal(t, "Apollo 11 Mission Report",
        dto.Title) // MUST fail before fix
    assert.Nil(t, dto.Author)                       // acceptable
    assert.Nil(t, dto.Category)                     // acceptable
    assert.Nil(t, dto.TorrentData)                  // acceptable
}
```

Falsifiability rehearsal (§6.N): Before applying the fix, run this test. It **MUST** fail with:

```
assert.Equal(t, "WP0-123456", dto.Id)
    expected: "WP0-123456"
    actual:   ""
```

7.2 Kotlin deserialization test (client side, reproduce the real WPO-* payload shape)

The test MUST use the *real 6-field-missing WPO- payload shape** (NOT the commentsPage-only fixture added in 1073).

File: core/network/api/src/test/kotlin/.../
TopicPageDtoDeserializationTest.kt

```
@Test
fun `WPO topic absent fields – five missing keys – throws before
    fix`() {
    // This is the REAL crash payload shape: commentsPage present
    // (1073 fix),
    // but id, title, author, category, torrentData ALL absent.
    val json = """"{"commentsPage":{"page":1,"pages":1,"posts":
        []}}""""

    // Before fix: throws MissingFieldException for "id".
    // After full fix (server returns all keys): this test becomes
    // a guard that
    // the server-side fix is in place (test should pass with the
    // proper response).
    val ex = assertThrows<SerializationException> {
        Json.decodeFromString<TopicPageDto>(json)
    }
    assertThat(ex.message).contains("id") // Primary assertion:
        correct field named
}

@Test
fun `WPO topic proper IA response – id and title populated –
    deserializes correctly`() {
    // This is the EXPECTED payload shape AFTER Option A fix:
    val json = """"{"id":"WPO-123456","title":"Apollo 11 Mission
        Report",
        "author":null,"category":null,"torrentData":null,
        "commentsPage":{"page":1,"pages":1,"posts":[]}}""""

    val dto = Json.decodeFromString<TopicPageDto>(json)

    // PRIMARY assertions on user-visible fields:
    assertThat(dto.id).isEqualTo("WPO-123456")
    assertThat(dto.title).isEqualTo("Apollo 11 Mission Report")
    assertThat(dto.author).isNull()
    assertThat(dto.torrentData).isNull()
}
```

Falsifiability rehearsal for the first test: The test already fails today (before any fix) because `Json.decodeFromString<TopicPageDto>` on the 5-absent-field payload throws `MissingFieldException` for `id`.

Confirm: Mutation: use the real absent-field payload. Observed: SerializationException: Field 'id' is required for type with serial name 'TopicPageDto', but it was missing. Reverted: yes.

8. Three Strongest File:Line Citations

1. **lava-api-go/internal/handlers/handlers.go:77** var _ ScraperClient = (*rutracker.Client)(nil) — compile-time proof that only *rutracker.Client ever handles /topic2. No provider dispatch is architecturally possible without changing this seam.
 2. **lava-api-go/internal/archiveorg/provider.go:118-137** GetTopic(ctx, id) returns a *provider.TopicResult with real ID and Title from archive.org metadata. The data exists and is fetchable; it is simply unreachable from the /topic2/:id route today.
 3. **core/network/api/src/main/kotlin/lava/network/dto/topic/TopicPageDto.kt:7-14** val id: String and val title: String have no defaults; val author: AuthorDto?, val category: CategoryDto?, val torrentData: TorrentDataDto? are nullable but ALSO have no defaults — all five are required keys in JSON per kotlinx.serialization. Absent keys throw MissingFieldException. Only commentsPage has a default (1073 partial fix). The remaining 5 fields are the crash surface.
-

9. One-Line Recommendation

Option (a) — server-side population: add a numeric-ID gate at handlers/topic.go after `id := c.Param("id")` that returns 404 for non-integer IDs (immediate hotfix), then wire the existing `archiveorg.GetTopic` at `internal/archiveorg/provider.go:118` into the `GetTopicPage` handler for WPO-* IDs to return real title + id from archive.org metadata; this is \$6.J-clean because the `rutracker` path is completely unchanged and masking a genuine missing-field error on a normal `rutracker` topic is impossible.